

TRADING PLATFORM

Arena Developer Master Guide + Local Development Setup

Professional Indian market analytics, F&O scanners and realtime trading-terminal style dashboard

Prepared for Arena as the primary AI developer

This document tells Arena what to build, how to structure it, and how to work safely in phases.

Contents

1. Project status and immediate workflow 3
2. Product objective 4
3. Technology architecture 5
4. Repository structure 6
5. Authentication and security 7
6. Global UI and dashboard 8
7. Market ticker requirements 9
8. Core modules overview 10
9. Market Movement / Sector Heatmap 11
10. BTST Scanner 12
11. Intraday Boosters 14
12. 15-Min Breakout 15
13. FII / DII 17
14. Index Mover and Global Markets 18
15. Market-data architecture 19
16. Database and historical data 20
17. Realtime/WebSocket and market session engine 21
18. Scanner engine and false-signal handling 22

19. API, testing and production	23
20. Phase-by-phase build order	24
21. Arena operating rules	25
22. Master prompt for Arena	26

1. Project Status and Immediate Workflow

The developer machine setup is already complete enough to start the application. The product owner is building the application locally with Next.js and using Arena as the primary AI developer. The local environment is intentionally kept simple; Docker/WSL/Redis are not mandatory for the initial build and can be introduced when the realtime/scanner workload requires them.

Component	Status	Purpose
VS Code	Ready	Local coding and repository management
Node.js + npm	Ready	Run Next.js/NestJS applications
Git + GitHub	Ready	Version control and remote source storage
PostgreSQL	Ready	Primary application database
Next.js frontend	Ready	Current local app at localhost:3000
Docker / WSL	Deferred	Add only when required
Redis	Deferred	Add when realtime cache/queue architecture needs it
Arena	Primary AI developer	Inspect, plan, implement, test, document

Current local application

```
C:\Projects\trading-platform\web
Local URL: http://localhost:3000
Database: trading_platform
Database user: postgres
PostgreSQL port: 5432
```

Core rule: Do not ask Arena to build the entire system in one uncontrolled pass. Arena should inspect the repository first, then implement one phase at a time.

2. Product Objective

Build a professional dark-mode web trading terminal for Indian markets, focused on market intelligence, F&O scanner workflows, options context, realtime market movement and historical institutional activity. The platform is a decision-support and analytics product; it is not a broker order-execution system in the current scope.

- **Primary focus:** NSE market intelligence, F&O stocks, options, sectors, indices and market context.
- **Independent modules:** each scanner has its own universe, rules, confirmation, invalidation and output.

- **Realtime:** market data must flow from a backend/data layer to the browser through a controlled server-side architecture.
- **No invented production data:** mock data may be used only in clearly separated development/demo mode.
- **No guaranteed signals:** the system may filter weak setups, but it cannot promise zero false signals.

3. Technology Architecture

```

Browser
|
v
Next.js Frontend
|
v
NestJS API / Services
|
+--> Authentication
+--> Market Data Service
+--> Options Service
+--> FII/DII Service
+--> Scanner Engine
+--> Historical Data Service
+--> User/Settings Service
|
+--> PostgreSQL
+--> Redis (later, when required)
+--> Licensed Market Data Provider

```

Technology choices

- **Frontend:** Next.js + React + TypeScript + Tailwind CSS + App Router.
- **Backend:** Node.js + NestJS + TypeScript.
- **Database:** PostgreSQL.
- **ORM:** Prisma.
- **Realtime:** WebSocket / Socket.IO or equivalent server-side realtime architecture.
- **Version control:** Git + GitHub.
- **Package manager:** npm for the first implementation because it is already configured locally.

4. Repository Structure

```

trading-platform/
apps/
  web/
  api/
  worker/
  market-data/

packages/
  database/
  types/
  indicators/
  scanner-engine/
  ui/
  config/

```

```
prisma/
docs/
scripts/
```

The exact structure can be simplified while the project is small, but the responsibilities must remain separated. Frontend UI should not contain scanner calculations. Market data normalization should not be mixed with presentation code. Background processing should be able to move to a worker process later without rewriting the UI.

5. Authentication and Security

- Login, registration, logout, forgot-password and reset-password flows.
- Protected application routes and secure session handling.
- Future-ready role support: Admin, Standard User, Premium User.
- Secure password hashing; never store plaintext passwords.
- HTTP-only secure cookies or an equivalent secure session mechanism.
- Input validation and rate limiting.
- Never expose database credentials or private market-data credentials to the browser.
- Use .env / .env.example for secrets and configuration.
- Audit logging for important account and admin actions.

Example configuration categories:

```
DATABASE_URL=
API_URL=
NEXT_PUBLIC_API_URL=
SESSION_SECRET=
REDIS_URL=
MARKET_DATA_PROVIDER=
MARKET_DATA_API_KEY=
MARKET_DATA_API_SECRET=
```

6. Global UI and Dashboard

The UI should look like a professional trading terminal rather than a generic SaaS landing page. Use dark navy/black backgrounds, slightly lighter cards, electric blue accents, green positive values, red negative values, thin borders, dense tables and compact information hierarchy.

Desktop shell:

```
+-----+
| Sidebar | Indian ticker row          |
|         | Global ticker row          |
|         | +-----+
|         | Main dashboard / module content |
|         |                               |
+-----+
```

Sidebar

Home dashboard card grid must explicitly include: Market Movement / Sector Heatmap, BTST Scanner, Intraday Boosters, 15-Min Breakout, FII/DII, Index Mover and Global Markets. Each card should have an icon, module name, short description, status/freshness state, and an Open action.

- Home
- Market -> Market Movement, Sector Heatmap, Index Movement

- Scanners -> BTST Scanner, Intraday Boosters, 15-Min Breakout
- Markets -> FII/DII, Global Markets
- Watchlist, Alerts, Trading Journal
- Settings and User/Profile

7. Two-Row Market Ticker

Row 1 - Indian markets

- NIFTY 50
- BANK NIFTY
- FINNIFTY
- NIFTY MIDCAP
- NIFTY SMALLCAP
- NIFTY IT
- NIFTY AUTO
- NIFTY PHARMA
- NIFTY FMCG
- other configured NSE indices
- BTC/USD

Row 2 - Global markets

- DOW JONES
- S&P 500
- NASDAQ
- FTSE London
- DAX
- GIFT NIFTY
- NIKKEI 225
- HANG SENG
- BTC/USD only if not duplicated in row 1

Each ticker item should show: name, current value, absolute change, percentage change, positive/negative state, market status and data freshness/delay status when available.

8. Core Modules Overview

Requirement coverage audit: the finalized scope contains seven explicitly defined modules/features: Market Movement / Sector Heatmap, BTST Scanner, Intraday Boosters, 15-Min Breakout, FII/DII, Index Mover and Global Markets. The three strategy scanners keep their original independent logic and option drill-down rules. Screenshot-only labels that were not separately defined as requirements are not treated as finalized functionality.

Module	Purpose
Market Movement / Sector Heatmap	Market intelligence and drill-down
BTST Scanner	End-of-day F&O option setup scanner
Intraday Boosters	Realtime +/-3% F&O movers with option context
15-Min Breakout	Confirmed 15-minute breakout/breakdown scanner

FII/DII

Daily institutional activity with range controls

Index Mover

Stocks contributing to index movement

Global Markets

Global index/crypto context

9. Market Movement / Sector Heatmap

This is a market-intelligence module, not primarily a trade-signal module.

Market
 -> Index
 -> Sector
 -> Stock

- Live sector strength distribution centered on 0: weak sectors on the left, strong sectors on the right.

Add a visible live market-session progress indicator running from 09:15 to 15:30 IST. It should show elapsed session progress and current market state without becoming a trade signal.

Index/sector universe must include at minimum NIFTY 50, BANK NIFTY, FINNIFTY, NIFTY MIDCAP, NIFTY SMALLCAP and configured NIFTY sector indices. Constituent membership should come from the instrument/index master rather than hard-coded lists scattered across the UI.

- Sector heatmap with block intensity showing relative strength/weakness.
- Sector blocks can vary by importance/activity.
- Click sector -> show constituent stocks.
- Click stock -> detailed stock context.
- Useful fields: LTP/CTP, percent change, volume, relative volume, breadth, OI where relevant, momentum and relative strength.

10. BTST Scanner

F&O-only end-of-day scanner. Do not treat every green/red stock as a setup.

Tabs

- Top BTST
- Bullish Reversal -> CE
- Bullish Momentum -> CE
- Bearish Reversal -> PE
- Bearish Momentum -> PE

Bullish Reversal -> CE

- Stock weakness followed by recovery
- Near meaningful support
- Selling exhaustion / recovery
- 2:30-3:00 closing strength when relevant
- Volume and relative volume
- Futures OI / OI acceleration where meaningful
- Sector and broader market context

Bullish Momentum -> CE

- Strong upward movement
- Strength into close
- Volume confirmation
- Sector/market support
- Near day high where relevant
- Derivatives/OI context

Bearish setups

- Bearish reversal: prior strength, resistance, weakening/reversal, volume/OI and context.
- Bearish momentum: strong downside, support break, weak sector, volume/OI and broader market context.

Scoring

Top BTST must combine and rank the strongest qualifying results across all four setup tabs; it must not introduce a fifth unrelated strategy.

The BTST engine should refresh during the late-session window and provide a final ranking around the 15:00 area, while keeping the exact evaluation window configurable.

A 0-100 ranking score may be used as an aggregation tool, but it is not certainty. Weights must be configurable and backtestable. Candidate inputs include sector strength, relative stock strength, price structure, relative volume, futures OI, OI acceleration, market regime, liquidity and option quality.

Option selection

- CE or PE
- ATM
- ITM1
- ITM2
- Strike
- LTP/CTP
- Day High
- Day Low
- Volume
- OI
- OI change
- IV
- Bid/Ask
- Spread
- Expiry
- Lot size

Option-quality selection should additionally evaluate liquidity, spread, delta, IV, theta, expiry proximity and expected-move fit where the provider supplies these values. These factors are quality filters, not guaranteed outcome predictors.

Option rule: Never hard-code strike intervals. Use the actual listed contract/option-chain data for each underlying.

11. Intraday Boosters

Realtime F&O scanner for stocks moving strongly during the session.

- Universe: F&O stocks only.
- Triggers: +3% or higher OR -3% or lower.
- Views: Up Boosters and Down Boosters.
- Sort by distance from day high/low and relative volume.

Configurable filters should include +/-3%, +/-4% and +/-5% movement thresholds, near-Day-High/near-Day-Low proximity, minimum volume/relative-volume requirements and optional sector/market context.

This module surfaces candidates only. It must not automatically label a stock as BUY and must not place broker orders in the current scope.

Underlying fields

- Current price
- Percent change
- Day High
- Day Low
- Distance from extreme
- Volume
- Relative volume
- Status

Option drill-down

- Upside -> CE
- Downside -> PE
- Show ATM + ITM1 + ITM2
- Include LTP/CTP, Day High/Low, Volume, OI, IV, Bid/Ask, Strike, Expiry

Use the same option-quality context as BTST where available: liquidity, spread, delta, IV, theta and expected-move fit.

12. 15-Min Breakout Scanner

F&O-only scanner comparing the live/developing market against the previous completed 15-minute candle.

```
09:15-09:30
09:30-09:45
09:45-10:00
...
15:15-15:30
```

- Upside: compare current price with previous completed 15m candle high.
- Downside: compare current price with previous completed 15m candle low.
- Configurable near-breakout thresholds can be 0.10%, 0.25%, 0.50%, 1.00%.

For upside distance-to-level, use a deterministic formula such as $(\text{previousHigh} - \text{currentPrice}) / \text{previousHigh} * 100$. For downside distance, use $(\text{currentPrice} - \text{previousLow}) / \text{previousLow} * 100$. Keep the threshold configurable.

For every confirmed result, show the underlying CTP/LTP, previous 15-minute high/low, current distance from the breakout/breakdown level and the option drill-down. Use the same option field set as BTST: strike, LTP/CTP, day high, day low, volume, OI, OI change, IV, bid/ask, expiry and lot size.

- Statuses: Near, Triggered/Breakout, Confirmed, Invalidated.

Confirmation

- Price crosses level
- Volume confirmation
- Relative volume
- Candle-body strength
- Current candle structure
- Sector context
- Broader market context
- OI/derivatives confirmation where meaningful
- Option liquidity

False-breakout handling

- Break previous high then return below it -> invalidate the upside breakout.
- Break previous low then recover above it -> invalidate the downside breakdown.
- A touch is not a confirmed breakout.

Options

- Upside confirmed breakout -> CE: ATM, ITM1, ITM2.
- Downside confirmed breakdown -> PE: ATM, ITM1, ITM2.

Signal integrity: The system can aggressively filter weak setups, but it cannot truthfully guarantee zero false signals.

13. FII / DII Module

When the module opens, show only the latest 10 TRADING DAYS by default. The phrase "10 days" means trading days, not calendar days.

Date	FII Buy	FII Sell	FII Net	IN MARKET	DII Net	DII Buy	DII Sell
------	---------	----------	---------	-----------	---------	---------	----------

The daily FII/DII dataset is the primary source of truth; the 10-day opening view is simply the latest 10-trading-day filter over the same daily records.

Date filters

- 10 Days
- 1 Month
- 3 Months
- 6 Months
- Custom Range

When the user selects 1 Month, fetch only the records in that range. Do not load an entire year unnecessarily. Date calculations must respect weekends and exchange holidays.

14. Index Mover and Global Markets

Index Mover

- Selected index
- Constituent stock
- Index weight where available
- Price change
- Point contribution
- Percent contribution
- Volume
- Direction
- Sort by positive or negative contribution

Global Markets

- DOW JONES
- S&P 500
- NASDAQ
- FTSE London
- DAX
- GIFT NIFTY
- NIKKEI 225
- HANG SENG
- BTC/USD

Show current value, absolute change, percentage change, market status, last update and data freshness/delay status. Never label delayed data as realtime.

15. Market-Data Architecture

```
interface MarketDataProvider {
    getQuote(symbol)
    getQuotes(symbols)
    getOptionChain(symbol)
    getHistoricalCandles(symbol, range)
    getInstrumentMaster()
    subscribe(symbols)
    unsubscribe(symbols)
}
```

- Implement provider adapters so the market-data vendor can be replaced.
- Support development MOCK mode and production provider mode.
- Normalize symbols, instruments, expiries, strikes, option types, prices, volume, OI, OHLC and timestamps.
- Do not connect every browser directly to the market-data vendor.

16. Database and Historical Data

Core entities:
users
roles
sessions

```

user_settings

instruments
stocks
indices
sectors
futures
options
contracts

market_ticks
ohlc_1m
ohlc_5m
ohlc_15m
daily_ohlc

fii_dii_daily

sector_snapshots
index_snapshots

scanner_results
scanner_events
btst_results
intraday_boosters
breakout_results

watchlists
alerts
audit_logs

```

Use Prisma migrations for schema changes. Do not manually create application tables in pgAdmin. Index high-frequency query fields such as symbol, timestamp, trading date, expiry, strike and scanner type.

Historical storage strategy

- Use cache/live state for realtime state.
- Use PostgreSQL for durable history.
- Do not store unlimited raw ticks without a retention/storage plan.
- Create daily aggregation and cleanup policies where appropriate.

17. Realtime, WebSocket and Market Session Engine

```

Market Data Provider
|
v
Ingestion / Normalizer
|
+--> Cache / persistence
|
+--> Scanner engines
|
v
WebSocket Gateway
|

```

v
Next.js Browser

- WebSocket reconnect handling
- Heartbeat/ping
- Stale connection detection
- Last-known value handling
- Data freshness indicator
- Clear "Reconnecting" / "Data delayed" state

Central market session service

- Pre-market
- Market open
- Continuous session
- Market close
- Weekend
- Exchange holiday
- Unavailable-data state

Indian cash-market session logic should be centralized around 09:15-15:30 IST, while holiday/weekend checks come from a configurable exchange calendar. Do not duplicate market-clock code in each module.

18. Scanner Engine and False-Signal Handling

Each scanner should be a deterministic engine with a clear universe, inputs, conditions, score (when used), confirmation rules, invalidation rules and output schema.

```
ScannerResult
- scannerType
- symbol
- direction
- status
- score
- triggeredAt
- confirmationState
- invalidationReason
- supportingMetrics
```

- No setup
- Watch
- Triggered
- Confirmed
- Invalidated

Store invalidation reasons where possible. This makes later backtesting and performance analysis far easier.

Backtesting-ready design: scanner rules must be deterministic and replayable on historical data. BTST evaluation should use actual option prices where available, not only the underlying. Future backtests must account for brokerage, taxes/fees, slippage, liquidity/spread effects and multiple market regimes. Score weights and confirmation thresholds must remain configurable.

19. API, Testing and Production

Example API surface

```

POST /auth/login
POST /auth/register
POST /auth/logout

GET /market/indices
GET /market/global
GET /market/sectors
GET /market/stocks/:symbol
GET /options/:symbol

GET /scanners/btst
GET /scanners/intraday-boosters
GET /scanners/15m-breakout

GET /fii-dii
GET /index-movers

```

Testing layers

- Unit tests: scanner conditions, breakout distance calculations, trading-day calculations, scores.
- Integration tests: API, database, authentication, provider adapter.
- Browser tests: login, dashboard, scanners, FII/DII date range, WebSocket reconnect and market-closed states.

Production hardening

- Database backups
- Authentication/security review
- WebSocket stability
- Provider failure handling
- Logging and monitoring
- Error tracking
- Rate limiting
- Secret management
- Holiday/session correctness
- Stale-data indicators
- Performance and index review

20. Phase-by-Phase Build Order

Phase	Deliverable	Focus
Phase 1	Repository inspection and architecture	Do not modify until Arena has inspected the current repo.
Phase 2	Frontend application shell	Sidebar, header, tickers, dashboard cards, responsive shell.
Phase 3	NestJS backend	API structure, configuration, health endpoints, error handling.

Phase 4	PostgreSQL + Prisma	Connection, schema foundation, migrations.
Phase 5	Authentication	Login, registration, sessions, protected routes.
Phase 6	Market-data abstraction	Mock provider, normalized models, provider adapter interface.
Phase 7	Market Movement	Sector/index drill-down and heatmap.
Phase 8	Index Mover	Index contribution view.
Phase 9	Intraday Boosters	+/-3% F&O movers and options drill-down.
Phase 10	15-Min Breakout	Candle engine, confirmations and invalidation.
Phase 11	BTST	Four setup tabs + Top BTST + option selection.
Phase 12	FII/DII	10 trading day default + date ranges.
Phase 13	Global Markets	Global ticker/page and freshness state.
Phase 14	Realtime + live provider	WebSockets and licensed provider adapter.
Phase 15	Testing and hardening	Unit/integration/browser tests and production review.

21. Arena Operating Rules

1. Inspect the repository before making architectural changes.
2. Do not delete or replace large portions of the existing code without explaining the impact.
3. Implement one phase at a time.
4. Keep commits small and meaningful.
5. Use TypeScript strictly and clear naming.
6. Create reusable components and avoid duplicated business logic.
7. Keep scanner business logic out of React components.
8. Never invent production market data.
9. Never hard-code option strike intervals.
10. Never treat a touch as a confirmed breakout.
11. Never promise zero false signals.
12. Distinguish mock, delayed and realtime data states.
13. Keep secrets out of source code and public frontend variables.
14. At the end of each phase report files changed, dependencies, database changes, APIs, test steps and remaining work.

22. Master Prompt for Arena

Copy the following prompt into Arena Agent/Code Arena after the GitHub repository is connected. The purpose of this first prompt is to force a repository inspection and controlled Phase 1 implementation rather than an uncontrolled one-shot build.

You are the lead full-stack developer and software architect for a professional web-based Indian stock market analytics and F&O scanner platform.

I want you to build this application as a production-oriented application, not a demo.

IMPORTANT:

- Do not create a generic stock-market website.
- Do not simplify or remove the modules specified in this project brief.
- Do not invent market data.
- Use mock/demo data only in clearly separated development mode.
- Keep each scanner/module independent.
- Do not combine unrelated strategies.
- No automatic broker order execution is in the current scope.
- This is a market analytics and decision-support platform.
- No scanner can guarantee zero false signals.

TECH STACK

Frontend: Next.js, React, TypeScript, Tailwind, App Router.

Backend: Node.js, NestJS, TypeScript.

Database: PostgreSQL.

ORM: Prisma.

Realtime: WebSocket/Socket.IO or equivalent.

Version control: Git + GitHub.

Package manager: npm.

ARCHITECTURE

Browser -> Next.js -> NestJS services -> PostgreSQL / optional Redis / market-data provider.

The market-data provider must be isolated behind a MarketDataProvider interface. The browser must not connect directly to the provider.

FRONTEND

- Home dashboard cards: Market Movement / Sector Heatmap, BTST Scanner, Intraday Boosters, 15-Min Breakout, FII/DII, Index Mover and Global Markets.

Build a professional dark trading-terminal interface with:

- left sidebar
- two-row top market ticker
- responsive main workspace
- module cards
- dense market tables
- green/red movement states

SIDEBAR

Home

Market -> Market Movement, Sector Heatmap, Index Movement

Scanners -> BTST, Intraday Boosters, 15-Min Breakout

Markets -> FII/DII, Global Markets

Watchlist, Alerts, Trading Journal

Settings, Profile

TWO-ROW TICKER

Row 1: NIFTY 50, BANK NIFTY, FINNIFTY, NIFTY MIDCAP, NIFTY SMALLCAP, NIFTY IT, NIFTY AUTO, NIFTY PHARMA, NIFTY FMCG, other configured Indian indices, BTC/USD.

Row 2: DOW JONES, S&P 500, NASDAQ, FTSE London, DAX, GIFT NIFTY, NIKKEI 225, HANG SENG.

Each item should show name, current value, absolute change, percent change and freshness/status.

MARKET MOVEMENT / SECTOR HEATMAP

- Include a visible live market-session progress indicator from 09:15 to 15:30 IST.
- Minimum index/sector universe: NIFTY 50, BANK NIFTY, FINNIFTY, NIFTY MIDCAP, NIFTY SMALLCAP and configured NIFTY sector indices, using instrument/index master data.

Purpose: market intelligence, not a primary trade signal.

Hierarchy: Market -> Index -> Sector -> Stock.

Show live strength distribution centered around zero, sector heatmap, constituent stocks, percent change, volume, relative volume, breadth and useful derivatives context.

BTST

F&O stocks only.

Tabs: Top BTST, Bullish Reversal CE, Bullish Momentum CE, Bearish Reversal PE, Bearish Momentum PE.

Use price structure, support/resistance, volume, relative volume, sector, broader market, futures OI and close strength where relevant.

Provide ranking/score only as a ranking mechanism.

Top BTST must aggregate and rank the strongest qualifying results across the four setup tabs. It is not a separate fifth strategy.

Option quality may consider liquidity, spread, delta, IV, theta, expiry proximity and expected-move fit where available.

Options: CE/PE, ATM, ITM1, ITM2 with strike, LTP, day high/low, volume, OI, OI change, IV, bid/ask, expiry and lot size.

Never hard-code strike intervals.

INTRADAY BOOSTERS

F&O stocks only.

Trigger: +3% or higher OR -3% or lower.

Show day high/low, distance from extreme, volume and relative volume.

Support configurable movement thresholds (+/-3%, +/-4%, +/-5%), near-Day-High/near-Day-Low filters and minimum volume/relative-volume filters.

This module is informational/candidate-focused and must not automatically place trades or declare a guaranteed BUY signal.

Upside drills into CE, downside drills into PE.

Show ATM, ITM1, ITM2.

15-MIN BREAKOUT

F&O stocks only.

Use previous completed 15-minute candle.

Use deterministic distance calculations to the prior high/low and configurable near-breakout thresholds such as 0.10%, 0.25%, 0.50% and 1.00%.

For confirmed results, display the underlying CTP/LTP, prior 15-minute high/low, distance to the level and option drill-down with ATM/ITM1/ITM2 plus strike, LTP/CTP, day high/low, volume, OI, OI change, IV, bid/ask, expiry and lot size.

Statuses: Near, Triggered/Breakout, Confirmed, Invalidated.

Confirmation: price cross, volume, relative volume, candle strength, sector, market context, OI where meaningful, option liquidity.

False breakout: if price breaks and quickly returns through the level, invalidate it.

Do not treat a touch as confirmed.

FII/DII

- Exact table fields: Date, FII Buy, FII Sell, FII Net, IN MARKET, DII Net, DII Buy, DII Sell.

- Daily records are the primary dataset; the default 10-day view is a latest-10-trading-day filter.

Default: latest 10 trading days.

Filters: 10 days, 1 month, 3 months, 6 months, custom.

Respect weekends and exchange holidays.

Fields: Date, FII Buy, FII Sell, FII Net, IN MARKET, DII Net, DII Buy, DII Sell.

Fetch only the selected range; do not load a full year unnecessarily.

INDEX MOVER

Show stocks contributing positive/negative points to selected indices, with weight where available.

GLOBAL MARKETS

DOW JONES, S&P 500, NASDAQ, FTSE London, DAX, GIFT NIFTY, NIKKEI 225, HANG SENG, BTC/USD.

Clearly label realtime vs delayed data.

DATABASE

Use PostgreSQL + Prisma.

Core entities should include users, roles, sessions, settings, instruments, stocks, indices, sectors, futures, options, contracts, market ticks, OHLC tables, FII/DII, sector/index snapshots, scanner results/events, watchlists, alerts and audit logs.

Use migrations. Do not manually create application tables in pgAdmin.

REALTIME

Use a central market session service and a server-side WebSocket layer.

Handle reconnects, stale data, heartbeats, market-closed state and data freshness.

Indian cash-market session logic: 09:15 to 15:30 IST, with exchange holiday support.

SECURITY

Secure passwords, protected routes, rate limiting, validation, secure cookies/session strategy, audit logging and environment variables. Never expose database/API secrets to the browser.

TESTING**BACKTESTING-READY DESIGN**

Scanner rules must be deterministic and replayable on historical data. BTST evaluation should use actual option prices where available and account for brokerage, taxes/fees, slippage, liquidity/spread and different market regimes. Keep score weights and thresholds configurable.

Unit tests for scanner calculations and date logic.

Integration tests for auth, APIs, database and provider adapters.

Browser tests for login, dashboard, scanner pages, FII/DII date filters and WebSocket reconnect behavior.

DEVELOPMENT PROCESS

1. Inspect repository.
2. Report current structure and versions.
3. Propose final folder architecture.
4. Identify what must be added for NestJS, Prisma and environment configuration.
5. Do not delete existing files.
6. Implement only Phase 1 after inspection.
7. At the end of every phase report changed files, dependencies, database changes, APIs, test steps and remaining work.
8. Keep commits small.
9. Do not build all scanners in one pass.

FIRST TASK

Inspect the existing repository and report:

- current Next.js/React/TypeScript setup
- current folder structure
- package dependencies
- routes/pages
- styling setup
- what is already implemented
- recommended production folder structure
- what is required for NestJS backend
- what is required for PostgreSQL/Prisma
- environment/configuration requirements

Do not modify anything until the inspection is complete.

After inspection, implement only Phase 1.

Final Handoff Checklist

- ☐ VS Code is installed and working.
 - ☐ Node.js + npm are working.
 - ☐ Git is installed and configured.
 - ☐ PostgreSQL is installed and the trading_platform database exists.
 - ☐ Next.js frontend runs at localhost:3000.
 - ☐ Arena is connected to the GitHub repository.
 - ☐ Arena has been given the master prompt.
 - ☐ Arena has inspected the repository before modifying it.
- ☐ Home page module cards match the finalized module list.
- ☐ Market Movement includes the 09:15-15:30 session progress indicator and required index universe.
- ☐ FII/DII defaults to 10 trading days and includes the IN MARKET field.
- ☐ 15-Min Breakout uses confirmation and invalidation states; a level touch is not a confirmed breakout.
- ☐ BTST Top BTST is an aggregate ranking of the four defined setup tabs.
- ☐ Implementation will proceed phase-by-phase.
 - ☐ Live market-data provider will be selected later and integrated through an adapter.